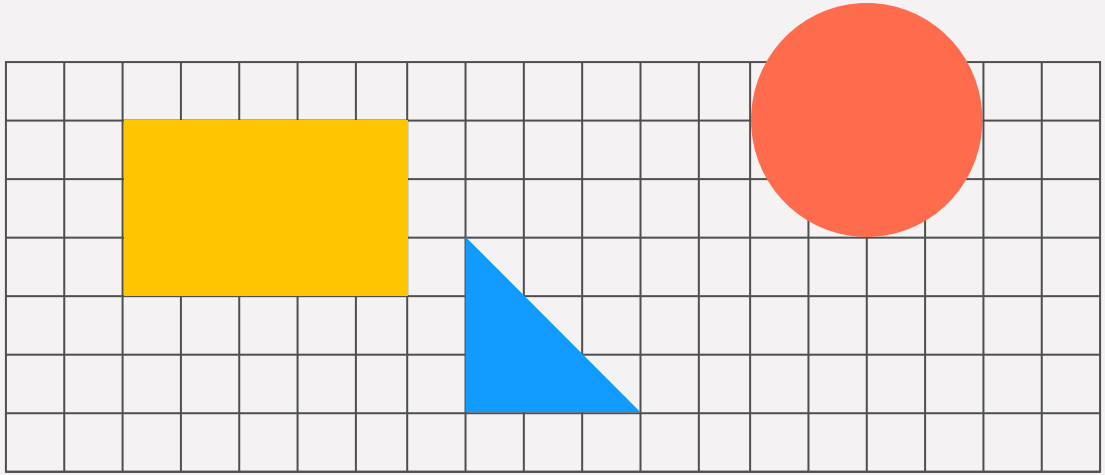
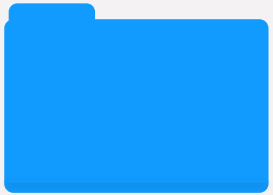
# DML- Data Manipulation Language



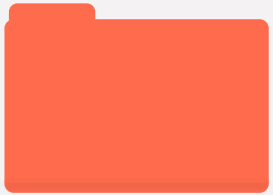
# DML Commands



INSERT-
Used to insert records into a table



UPDATE - used to modify records



DELETE - used to delete records

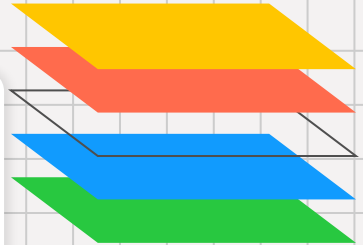


SELECT - used to retrieve records

# the fruits table

For the next examples, assume a fruit table with the following columns:

| fruits |
| --- |
| **fruitId** |
| fruitName |
| color |
| quantity |
| expirationDate |



```
CREATE TABLE fruit (
    fruitId        INTEGER PRIMARY KEY AUTOINCREMENT,
    fruitName      TEXT NOT NULL,
    color          TEXT,
    quantity       INTEGER NOT NULL,
    expirationDate TEXT
);
```

# INSERT specifying columns

Columns can be specified in any order
NOT NULL columns must be specified if there isn't a default value
Auto Increment columns do not need to be specified

I left out fruitId because it is a primary key set to auto-increment

```
-- Insert while naming columns (order doesn't matter)
INSERT INTO fruit (fruitName, color, quantity, expirationDate)
VALUES ('orange', 'orange', 3, '2020-11-03');

-- Verify new record
SELECT * FROM fruit;
```

BEST PRACTICE

You don't have to do all the columns if you are naming them

```
INSERT INTO fruit
(fruitName, quantity,
expirationDate)
VALUES ('orange', 3,
'2020-11-03');

Update to fill in missing
information:
UPDATE fruit
SET color = 'orange',
    expirationDate =
'2026-03-15'
WHERE fruitId = 1;
```

# INSERT WITHOUT specifying columns

Data must be entered for every field in the order of the columns
If there is no value for a column, null should be used
null should be used for Auto-Increment columns



```
-- Insert values in the order of the columns
INSERT INTO fruit VALUES
(NULL, 'apple', NULL, 3, '2020-11-03'),
(NULL, 'banana', 'yellow', 2, '2020-11-10');

-- Check results
SELECT * FROM fruit;
```

Without Column Names:

- Value order must match column order exactly
- You must enter data for the exact number of rows or sql will return an error
- Use ‘NULL’ for auto-incrementing default values, or where it is allowed.

# NULL !

Context:
In CREATE TABLE
Defines whether a column can ever be empty (NOT NULL = must have a value).
In INSERT
Represents “unknown,” “not applicable,” or “let the system handle it.”
In an AUTO_INCREMENT primary key
Tells the DB to generate the next number automatically.

NULL in the CREATE TABLE statement → a rule
When we write:

```
fruitName      TEXT NOT NULL,
```

we are saying:
"This column must always have a value — no empty or unknown entries allowed."
It’s a constraint — a rule that prevents bad data from being inserted.
So fruitName and quantity are required for every row.

NULL in the INSERT statement → a placeholder
When we insert NULL for a primary key (fruitId), it’s actually a signal to the database:
“You handle this — auto-generate the next ID.”

The AUTO_INCREMENT keyword makes SQL fill in the next available number automatically.
Example:

```
INSERT INTO fruit VALUES
(NULL, 'apple', NULL, 3, '2020-11-03');
```

The first NULL triggers auto-increment (fruitId gets 1, then 2…).
The second NULL (for color) means “unknown or not applicable.”
So, NULL in this case is not violating a constraint — it’s allowed because we didn’t declare color as NOT NULL

# UPDATE

IF YOU FORGET THE WHERE CLAUSE, ALL RECORDS WILL BE UPDATED !!!!

SYNTAX:

```
UPDATE tbl_name
SET field = value
WHERE condition
```

-- Basic UPDATE syntax

```
UPDATE fruit
SET color = 'red'
WHERE fruitName =
'apple';

-- View change
SELECT * FROM fruit;
```

# DELETE

IF YOU FORGET THE WHERE CLAUSE, ALL RECORDS WILL BE DELETED !!!!

SYNTAX:

```
DELETE FROM
tbl_name
WHERE condition
```

```
-- SAFE DELETE using WHERE

DELETE FROM fruit
WHERE fruitName =
'apple';

SELECT * FROM fruit;
```

# SELECT

IF YOU OMIT WHERE,
ALL RECORDS
WILL BE RETURNED

SYNTAX:

```
SELECT column1,
(column2-optional)
FROM tbl_name
WHERE condition;
```

**–Filtering SELECT with WHERE**

```
SELECT fruitName,
quantity
FROM fruit
WHERE quantity > 5;
```

returns data in fruitName and quantity from the fruit table that list quantity more than 5

```
SELECT * FROM fruit;
```

returns all column data from the fruit table

# the WHERE clause

defining which rows are affected or returned

controlling scope

- =
- <>
- >
- <
- LIKE
- IN
- IS NULL

WHERE filters rows.

```
SELECT fruitName, quantity
FROM fruit
WHERE quantity > 5;
```

*returns data in fruitName and quantity from the fruit table where quantity is greater than 5

| fruitId | fruitName | color | quantity |
|---|---|---|---|
| 1 | apple | red | 3 |
| 2 | apple | green | 5 |

- UPDATE fruit
  SET color = 'yellow'
  WHERE fruitName = 'apple';

safer pattern!

- UPDATE fruit
  SET color = 'green'
  WHERE fruitId = 1;

- SELECT * FROM fruit
  WHERE color IN (‘red’, ‘yellow’);

# JOIN (an intro.)

Combines rows from 2 (or more) tables

the default ‘inner join’ returns rows where values match

SYNTAX:

```
SELECT column1, column2
FROM table1
JOIN table2
ON table1.column = table2.column;
```

we want to know who purchased each fruit

**fruit**

| fruitId | fruitName |
|---|---|
| 1 | apple |
| 2 | banana |

**purchase**

| purchaseId | fruitId | customerName |
|---|---|---|
| 101 | 1 | Alex |
| 102 | 2 | Sam |

```
SELECT fruitName, customerName
FROM fruit
JOIN purchase
ON fruit.fruitId = purchase.fruitId;
```

**Result box**

| fruitName | customerName |
|---|---|
| apple | Alex |
| banana | Sam |